

V1 → V2 changes

What changed between `Expense_Receipt_Collector_v1.ipynb` (frozen baseline) and `Expense_Receipt_Collector_v2.ipynb`, and why. Written for the whole team, not just whoever built it — this is meant to be the source doc for Stage 7 (failure analysis) in the report.

Not in scope for this document: the 20 test cases, the 4 scoring criteria, and the scoring process. Those belong to whoever owns Stage 6/7 — this only covers what changed in the build.

Failures found in the V1 eval → fixes in V2

#	V1 failure	Root cause	V2 fix	Where
1	Model fabricated amounts instead of returning <code>null</code> for torn/illegible/missing totals	Prompt said "don't guess" but had no few-shot showing the null behaviour, and nothing checked the output afterward	Two hand-authored few-shot examples added to Module 1 (torn total → all-null, vague text → amount null); a code-level <code>extraction_status()</code> reads the returned JSON and reports <code>ok/partial/unable</code> so the review UI can visibly warn	Module 1 cell (few-shot + <code>extraction_status</code>)
2	Irrelevant line items on a receipt leaked into the description / confused the extraction	Module 1 was effectively reading the receipt as a list of items instead of one expense	Prompt now explicitly says: extract the grand total and merchant, do not enumerate or sum line items yourself	Module 1 system prompt
3	A hand-drawn / fake receipt was extracted and saved as if genuine	Nothing checked whether the image was a legitimate receipt before extracting from it	New Module 1a — Receipt Triage : a separate classification call (<code>printed</code> / <code>handwritten</code> / <code>hand_drawn</code> / <code>digital</code> / <code>not_a_receipt</code> / <code>unclear</code>) runs before extraction. A flagged result blocks saving until a human ticks an explicit override checkbox	New Module 1a cell; review UI
4	"How much did I spend this month?" returned nothing useful / looked broken once the calendar moved past the July–Aug data	Two compounding bugs: (a) the app had no notion of "today" at all, and (b) <code>answer_query</code> 's <code>start_date/end_date</code> parameters existed but the Query tab never actually populated them — variant C was silently searching the <i>entire</i> ledger regardless of what was asked	<code>TODAY</code> pinned (<code>2026-09-09</code> , see below); new rule-based <code>resolve_relative_range()</code> parses phrases like "this month" / "last week" / "August" into an exact date range with no LLM involved; that range is now actually wired into <code>answer_query</code> 's variant C; the system prompt requires stating the resolved range and saying so explicitly when it has no/partial data	Module 2 cell (<code>resolve_relative_range</code> , <code>answer_query</code>)

Design decisions made before building (team-approved)

- **TODAY pinned to 2026-09-09**, not the live clock. `DATA_MIN_DATE` / `DATA_MAX_DATE` stay derived live from whatever is in the ledger — never hardcoded — so the two update independently: "today" is stable for reproducible re-runs, "what data exists" always reflects the real ledger. *Rejected alternative*: pinning "today" to the ledger's own latest date — this was considered and turned down because it would make the today-is-ahead-of-the-data gap (the actual bug) impossible to occur, which defeats the point of the fix.
 - **Assume + state, not ask, for ambiguous dates.** Every resolvable time phrase gets a deterministic range and the model states it in the answer, rather than asking a clarifying question. This keeps every query single-turn (cheaper — a clarifying-question round trip roughly doubles token spend for that query — and keeps the eval harness's automated 20×3 run scoreable without a follow-up turn). A clarifying-question fallback for genuinely *unanchored* references (no date phrase at all) was intentionally **not** pre-built — if eval turns one up, that becomes a clean Stage 7 retest case rather than a manufactured one.
 - **Triage is a separate model call, not an extra field on Module 1's output.** The spec is strict that Module 1 returns exactly `{date, merchant, amount, description}`. Extending that schema would drift from spec; a separate Module 1a call keeps Module 1 untouched and gives the architecture diagram a clean new box.
 - **Mixed-category receipts:** one row, categorised by merchant as before — no splitting into multiple rows, no extra flag. Splitting was considered and rejected as exactly the complexity the spec already descoped.
 - **Flagged-receipt override:** a required checkbox ("I've reviewed this and confirm it's accurate despite the flag"), not a hard rejection. Chosen so the system still demonstrates graceful degradation + a human catch, rather than a dead end.
 - **Category dropdown is dynamic and appendable**, not fixed. It lists whatever is already in `category_lookup.xlsx` plus an "Other (type new)" option; picking Other appends the merchant → category mapping back into the Excel file. This is a second, explicit human-in-the-loop step — the human corrects the rule table, the AI still never suggests a category.
 - **Duplicate detection is a non-blocking warning**, not an auto-reject — a genuine repeat purchase is legitimate, so the human decides.
-

New in V2

- **Module 1a — Receipt Triage** (new cell): classifies image authenticity before extraction. **Known limitation:** its prompt has no real hand-drawn few-shot image yet — none of the sample receipts are hand-drawn. If the team has the actual failing image from the V1 eval, wiring it in as a few-shot pair will sharpen this considerably. Currently text-described only.
- **Review UI** now shows the receipt image (or text note) side-by-side with the editable fields — on every row, not just flagged ones — so a human can visually check a number against the source. `upload_and_review()` is the click-to-upload entry point (file picker + freeform-text fallback).
- `add_category_mapping()` persists human category corrections back to `category_lookup.xlsx`.
- `check_duplicate()` — code-level, no AI, warns on save.

- **Cost meter** (`CostMeter` / `METER`) wraps every OpenRouter call, prefers OpenRouter's actual billed cost when available (`extra_body={"usage": {"include": true}}`), falls back to a price table otherwise. Wired into the eval harness (`METER.reset()` before the run, `METER.summary()` after).
- `eval_outputs_v2_for_scoring.csv` — separate filename from V1's `eval_outputs_for_scoring.csv` so both sit in Drive for a side-by-side before/after comparison.

Function signature changes (if anything downstream calls these directly)

- `answer_query(question, variant="C", category=None)` — **dropped** `start_date` / `end_date` params from V1. They're resolved internally now via `resolve_relative_range()` instead of being passed in (and in V1 they were never actually being passed in by any caller anyway, which was part of bug #4).
- `extract_expense()` return shape is unchanged; `extraction_status()` is new and operates on its output rather than changing it.

Cost impact

Triage adds one extra Qwen-VL call per image receipt (~\$0.0005 each). Across a full 20×3 eval pass this adds a few cents at most — see `COST_FORECAST.md`; the "under \$5 for the whole project" estimate still holds comfortably.